

Data Structures and Algorithms

Class Test 2, May 7, 2026

Instructions:

- Please fill in your student number (and if possible name) correctly.
- Answer all questions by correctly filling the appropriate answer in the answer sheet.
- Please return only the answer sheet, and discard the question paper later.

Q1.

You are given a reference to a node B in a circular singly-linked list. B is neither the first nor the last node. You wish to delete node B . Which of the following strategies allows you to effectively “remove” the data of node B from the list in $O(1)$ time?

- (a) Traverse the list to find the node A such that $A.next = B$, then set $A.next = B.next$.
- (b) Copy the data from $B.next$ into B , then set $B.next = B.next.next$.
- (c) Set $B = B.next$.
- (d) It is impossible to delete node B in $O(1)$ time without a reference to the preceding node.

Answer: (b)

Explanation: Since you cannot find the predecessor in $O(1)$ time in a singly-linked list, the trick is to copy the data from the next node into the current node and then skip the next node.

Q2.

A sorting algorithm is considered *stable* if it preserves the relative order of records with equal keys. Which of the following statements regarding sorting stability is **false**?

- (a) Merge Sort is naturally stable because of how the merge step is typically implemented.
- (b) Insertion Sort is stable because swaps do not reorder equal elements.
- (c) Heap Sort is a stable sorting algorithm because it uses a structured binary tree.
- (d) Any unstable sort can be made stable by adding the original index of the element as a secondary key.

Answer: (c)

Explanation: Heap Sort is inherently unstable because the heap building and extraction processes involve swaps that can reorder equal elements across different branches of the tree.

Q3.

You are using a hash table of size 7 with the hash function $h(k) = k \bmod 7$. You use linear probing to resolve collisions. After inserting the keys 15, 22, and 8 in that order, what is the index of the key 8?

- (a) 1
- (b) 2
- (c) 3
- (d) 4

Answer: (c)

Explanation: 15 maps to index 1 ($15 \bmod 7 = 1$). 22 maps to index 1 ($22 \bmod 7 = 1$), but since it's full, it takes index 2. 8 maps to index 1 ($8 \bmod 7 = 1$); since 1 and 2 are full, it takes index 3.

Q4.

A specific binary tree has the following traversal sequences:

- **In-order:** D, B, E, A, F, C
- **Pre-order:** A, B, D, E, C, F

What is the **Post-order** traversal of this tree?

- (a) D, E, B, F, C, A
- (b) D, B, E, F, C, A
- (c) E, D, B, F, C, A
- (d) A, C, F, B, E, D

Answer: (a)

Explanation: Pre-order shows A is the root. In-order shows {D, B, E} are in the left subtree and {F, C} are in the right. Reconstructing the tree gives left child B (with children D, E) and right child C (with left child F). Post-order (Left, Right, Root) is D, E, B, F, C, A.

Q5.

A binary tree (not necessarily a complete binary tree) is stored in an array (starting at index 1). For any node at index i , its left child is at index $2i$ and its right child is at index $2i + 1$. [cite: 133] Given the array $[2, \text{empty}, 3, \text{empty}, \text{empty}, 15, 20, 18, 11]$, which index violates the binary tree property first?

- (a) Index 3
- (b) Index 6
- (c) Index 8
- (d) No violation exists.

Answer: (c)

Explanation: Index 8 is the left child of index 4 ($2 \times 4 = 8$). However, index 4 is *empty*. A node cannot have children if the node itself does not exist.

Q6.

Consider a dynamic array that starts with a capacity of 1. When the array is full, its capacity is doubled. If an insertion costs 1 unit plus the cost of copying elements during a resize, what is the **amortized** cost of n insertions?

- (a) $O(1)$
- (b) $O(\log n)$
- (c) $O(n)$
- (d) $O(n \log n)$

Answer: (a)

Explanation: Although occasional resizes are expensive ($O(n)$), they happen infrequently enough that the total cost for n insertions is $O(n)$, making the average (amortized) cost per insertion $O(1)$.

Q7.

You are given two sorted arrays A is of size m and B is of size n . You want to find the k -th smallest element in the combined set $A \cup B$. What is the best possible worst-case time complexity to achieve this?

- (a) $O(1)$
- (b) $O(\log(\min(m, n)))$
- (c) $O(\log(m + n))$
- (d) $O(\sqrt{n})$

Answer: (b)

Explanation: By using a binary search approach to partition the arrays, the k -th element can be found in logarithmic time relative to the smaller array. We need to find an element p in $A \cup B$ such that the total number of elements $< p$ in $A \cup B$ is k . Do a binary search with the middle element of the smaller array. Suppose A is the smaller array. Its middle element is greater than $\lfloor \frac{m}{2} \rfloor$ of the elements in A . Suppose the binary search gives an element at the p position of array B . Check whether $\lfloor \frac{m}{2} \rfloor + p == k$, then the p -th element of array B is the overall k -th element. If $\lfloor \frac{m}{2} \rfloor + p < k$, then choose the middle element of the right half of array A and recurse. If $\lfloor \frac{m}{2} \rfloor + p > k$, then choose the middle element of the left half of the array A and recurse.

Q8.

In a hash table using *Double Hashing*, the primary hash function is $h_1(k) = k \bmod 11$ and the secondary hash function is $h_2(k) = k \bmod 7$. The second hash function is used if the index from the first hash function is already occupied. If a collision occurs at $h_1(58)$, the value of $h_2(58)$ is used as the next index to check. What is the index of the **first** probe?

- (a) 3
- (b) 2

(c) 6

(d) 8

Answer: (b)

Explanation: $h_1(58) = 58 \bmod 11 = 3$. If index 3 is occupied, we check the index given by the secondary function. $h_2(58) = 58 \bmod 7 = 2$. Thus, the first alternative slot checked is 2.

Q9. Consider a min-heap containing n elements stored in an array. Which of the following statements is true?

- (a) The smallest element in the heap can be found in $O(\log n)$ time.
- (b) The height of a complete binary tree with n nodes is $O(\log n)$, ensuring that the worst-case time for a single insertion is $O(\log n)$.
- (c) Deleting the minimum element from the heap and then restoring the heap property takes $O(n)$ time in the worst case.
- (d) Any arbitrary unsorted array of size n can be treated as a valid min-heap without reordering the elements.

Answer: (b)

Explanation: Insertion involves adding a leaf and “bubbling up” to the root, which is limited by the tree’s height, $O(\log n)$. Option (a) is false ($O(1)$), (c) is false ($O(\log n)$), and (d) is false (heap property must be maintained).

Q10.

You are given a reference `curr` to a node in a doubly-linked list. Each node has `next` and `prev` references. You want to swap `curr` with its successor `curr.next`. Assuming `curr.next` is not null, which of the following code snippets correctly implements this swap?

- (a) (Incomplete logic)
- (b) (Correct logic)
- (c) (Incorrect pointers)
- (d) (Only partial swap)

Answer: (b)

Explanation: Option (b) correctly reassigns the four boundary pointers (the node before `curr` and the node after `succ`) and the pointers between `curr` and `succ` itself to swap their positions.

End of Paper